Universidad San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias y Sistemas
Intr. a la Programación y Computación 2
Ing. Daniel Gonzales

MANUAL TÉCNICO
PRÁCTICA #1

Henry Josué Argueta Champet
Reg. academico: 202131261
Fecha: 09/03/2026

ÍNDICE

Descripción.....3
Objetivos.....3
Tecnologías utilizadas.....4
Arquitectura del sistema.....4
Descripción de clases principales.....5
    Clase Main.....5
Diagrama Entidad-Relación.....5
Diagrama de Tablas.....6
Descripción General del Funcionamiento.....6
Control de Versiones.....7
Requisitos del Sistema.....7
Conclusión.....7

Descripción

Este documento ha sido creado con un propósito claro: servir como una guía detallada y completa para todos los colaboradores que deseen comprender, analizar y profundizar en el código fuente de nuestro programa. Mi objetivo es que, a través de esta explicación, puedan navegar por la estructura del proyecto con total confianza, entendiendo no sólo qué hace cada componente, sino también el porqué detrás de las decisiones de diseño y la lógica implementada.

Aquí, desglosamos cada sección del código, explicando su funcionalidad, las interacciones entre los módulos y los patrones de diseño utilizados. Esto no solo facilitará la incorporación de nuevos miembros al equipo, sino que también permitirá una colaboración más eficiente en futuras mejoras y actualizaciones.

Objetivos

Este manual técnico ha sido diseñado como una guía esencial para ayudar a comprender y dominar todos los aspectos del software. El objetivo principal es que, como usuario, sientas completamente la capacidad para utilizar, mantener y configurar el sistema de manera eficiente.

- Aplicar conceptos de bases de datos relacionales en el desarrollo de un videojuego.
- Aplicar programación orientada a objetos en la modelación de entidades del juego.
- Implementar interfaces gráficas interactivas utilizando Java Swing.
- Desarrollar la lógica necesaria para gestionar estados, puntajes y flujo de juego.

Tecnologías utilizadas

Tecnología	Descripción
Java	Lenguaje de programación utilizado para desarrollar la lógica del sistema
Maven	Herramienta para gestión de dependencias y construcción del proyecto
Git	Sistema de control de versiones
GitHub	Plataforma para almacenamiento del repositorio
IDE	(VS Code) Entorno de desarrollo utilizado

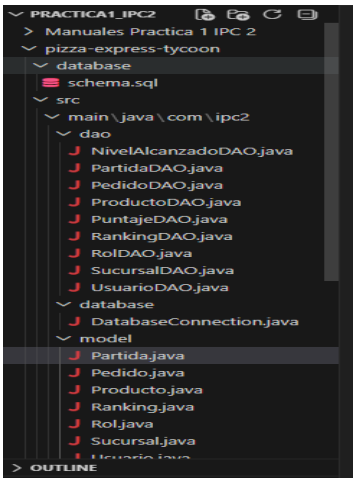
Arquitectura del sistema

El sistema sigue una estructura basada en programación orientada a objetos, donde cada entidad del sistema se representa mediante una clase.

La aplicación está organizada en diferentes componentes:

- Clases del modelo
- Clases de lógica
- Clase principal (Main)

Esto permite una mejor organización del código y facilita su mantenimiento.



Descripción de clases principales

Clase Main

La clase Main es el punto de inicio del programa.

- Funciones principales:
- Iniciar la aplicación
- Mostrar el menú principal

Controlar el flujo del sistema

```java
package com.ipc2.dao;

import com.ipc2.database.DatabaseConnection;
import com.ipc2.model.Usuario;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;

public class UsuarioDAO {

    public Usuario login(String nombreUsuario, String contrasena) {
        String sql = "SELECT id_usuario, nombre_usuario, nombre_completo, id_rol, id_sucursal, activo " +
                     "FROM usuarios WHERE nombre_usuario = ? AND contrasena = ? AND activo = true";

        try (Connection connection = DatabaseConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {

            statement.setString(1, nombreUsuario);
            statement.setString(2, contrasena);

```

Diagrama Entidad-Relación

El diagrama entidad-relación permite visualizar la estructura conceptual del sistema y la relación entre las diferentes entidades.

En el modelo se identifican entidades principales como:

- Usuario
- Rol
- Sucursal
- Producto
- Pedido
- Partida
- Puntaje
- Nivel

Las relaciones permiten establecer cómo interactúan las entidades dentro del sistema, por ejemplo:

- Un usuario pertenece a un rol.
- Un usuario trabaja en una sucursal.
- Una partida genera pedidos.
- Un pedido contiene productos.
- Una partida genera un puntaje.

Este modelo conceptual facilita la comprensión del funcionamiento del sistema antes de su implementación en la base de datos.

Diagrama de Tablas

El diagrama de tablas representa la estructura lógica de la base de datos del sistema.

Cada entidad del modelo conceptual se transforma en una tabla con sus respectivos atributos y claves.

Algunas de las tablas principales son:

- usuarios
- roles
- productos
- pedidos
- partidas
- puntajes
- niveles
- sucursales

Las relaciones entre tablas se establecen mediante claves foráneas, lo que permite mantener la integridad de los datos.

Por ejemplo:

- La tabla usuarios contiene una referencia al rol y a la sucursal.
- La tabla pedidos está relacionada con la partida.
- La tabla detalle_pedido conecta los productos con cada pedido.

Esto permite organizar la información de manera eficiente dentro del sistema.

Descripción General del Funcionamiento

El sistema inicia mediante la clase principal, la cual se encarga de iniciar el programa y controlar el flujo de ejecución.

Posteriormente, el sistema permite gestionar diferentes elementos como:

- creación de pedidos
- registro de productos
- gestión de partidas
- cálculo de puntajes

Cada operación se realiza mediante la interacción entre diferentes clases del sistema.

Control de Versiones

Durante el desarrollo del proyecto se utilizó Git como sistema de control de versiones.

Esto permite registrar los cambios realizados en el código a lo largo del desarrollo del sistema.

Los comandos principales utilizados fueron:

- git init
- git add .
- git commit -m "Cambios realizados"
- git push

Esto permite mantener un historial de versiones del proyecto.

```
PROBLEMS 2    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS
PS C:\Users\Pablo\OneDrive\Escritorio\Practica1_IPC2> git add .
PS C:\Users\Pablo\OneDrive\Escritorio\Practica1_IPC2> git commit -m "Correcion Manual Tecnico"
[main f95832c] Correcion Manual Tecnico
 28 files changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\Pablo\OneDrive\Escritorio\Practica1_IPC2> git push
Enumerating objects: 81, done.
Counting objects: 100% (81/81), done.
Delta compression using up to 8 threads
Compressing objects: 100% (37/37), done.
Writing objects: 100% (42/42), 49.69 KiB | 3.31 MiB/s, done.
Total 42 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 6 local objects.
To https://github.com/Henry18838/IPC2_Practica1_2026.git
   168a1dd..f95832c  main -> main
PS C:\Users\Pablo\OneDrive\Escritorio\Practica1_IPC2>
```

Requisitos del Sistema

Para ejecutar el sistema es necesario contar con:

- Java JDK 17 o superior
- Apache Maven
- Sistema operativo Windows, Linux o Mac

Conclusión

➔ El sistema Pizza Express fue desarrollado utilizando principios de programación orientada a objetos, lo que permite mantener una estructura organizada del código.
➔ El uso de diagramas de modelado facilita la comprensión del sistema y permite visualizar la relación entre las diferentes entidades.
➔ Además, el uso de herramientas como Git y Maven permite gestionar el proyecto de forma eficiente y mantener un control adecuado sobre las versiones del software.